



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 18

Dr. Asad Mahmood

Feb. 25, 2026

Lecture Outline

- Announcements ()
- Outline of Previous Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.4 Division
- Outline to Today's Lecture:
 - **Chapter 3 – Arithmetic for Computers**
 - 3.4 Division
 - 3.5 Floating Point

Chapter 3

Arithmetic for Computers

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

RISC-V Multiplication

- Four multiply instructions:
 - `mul`: multiply
 - Gives the lower 32 bits of the product
 - `mulh`: multiply high
 - Gives the upper 32 bits of the product, assuming the operands are signed
 - `mulhu`: multiply high unsigned
 - Gives the upper 32 bits of the product, assuming the operands are unsigned
 - `mulhsu`: multiply high signed/unsigned
 - Gives the upper 32 bits of the product, assuming one operand is signed and the other unsigned

Chapter Outline

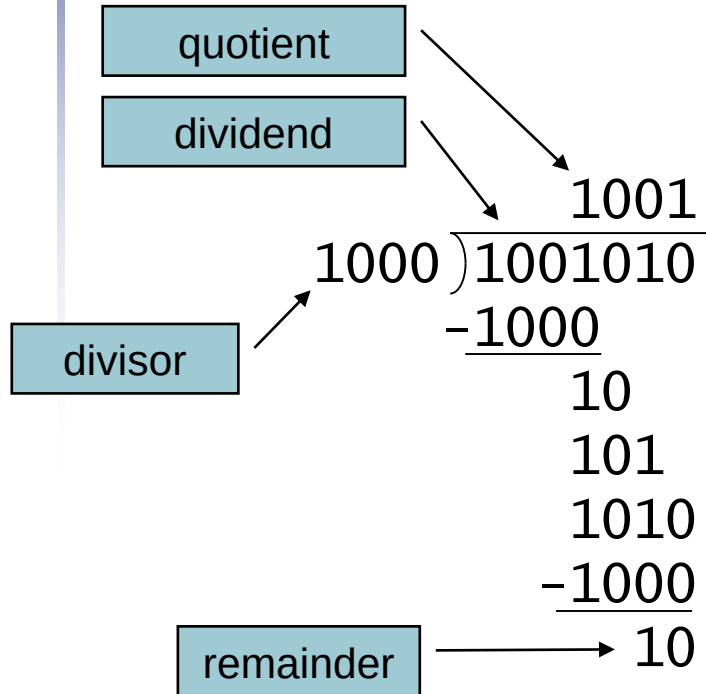
Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Division

*Divide et impera. Latin for
“Divide and rule,”
ancient political maxim cited by
Machiavelli, 1532*

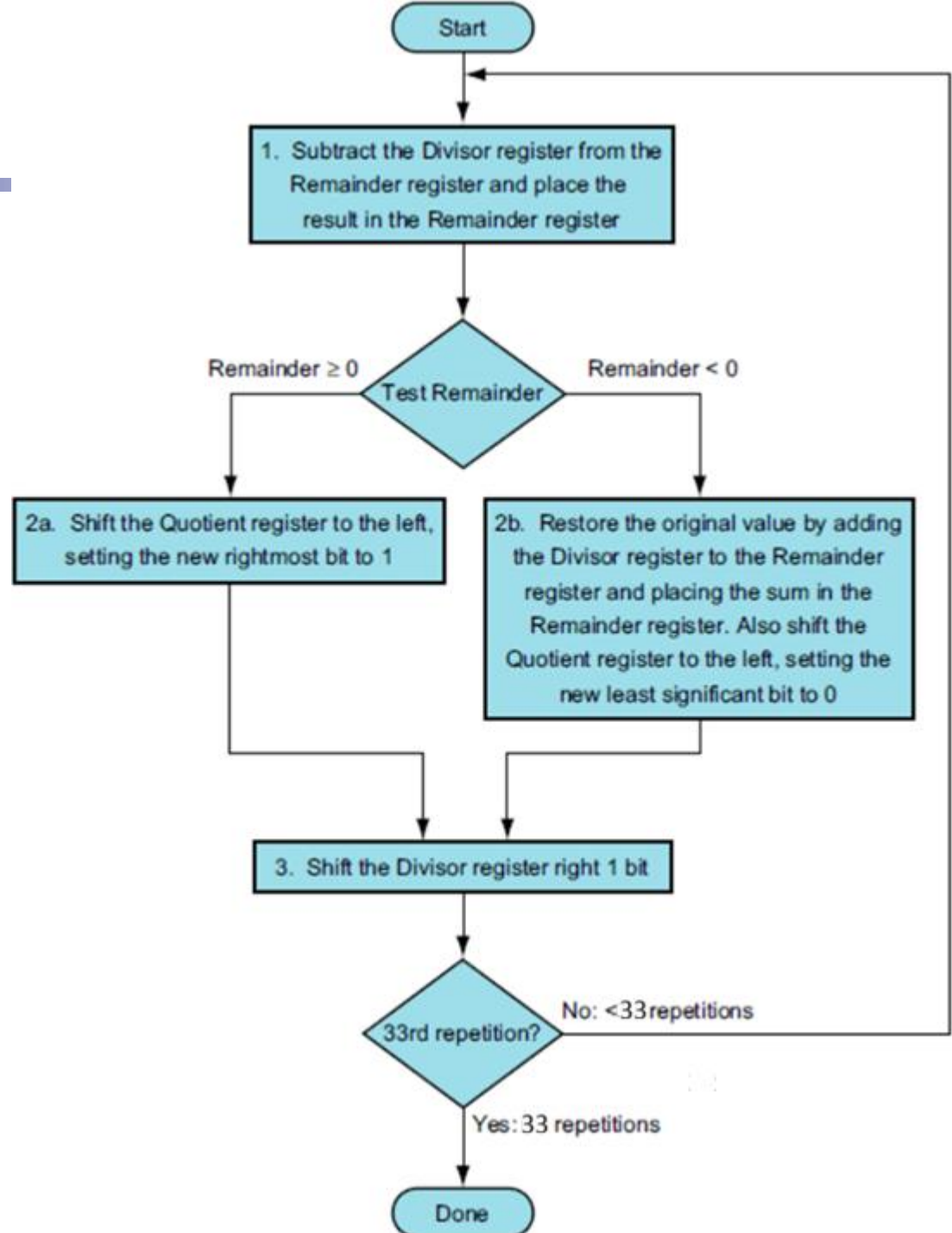
Division



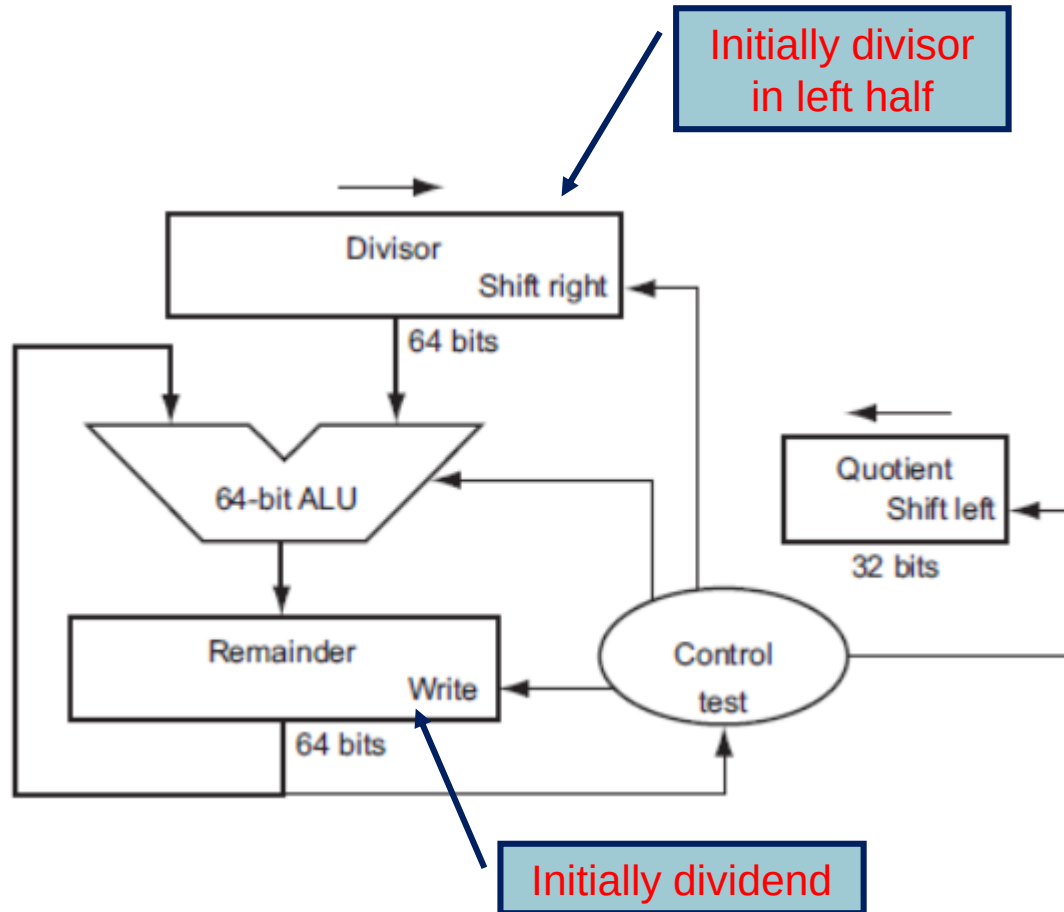
n-bit operands yield *n*-bit quotient and remainder

- More strange than multiplication
- Long division approach
 - If divisor $\leq$ dividend bits
 - 1 bit in quotient, subtract
 - Otherwise
 - 0 bit in quotient, bring down next dividend bit
- Restoring division
 - Do the subtract, and if remainder goes < 0 , add divisor back
- Signed division

Division Algorithm



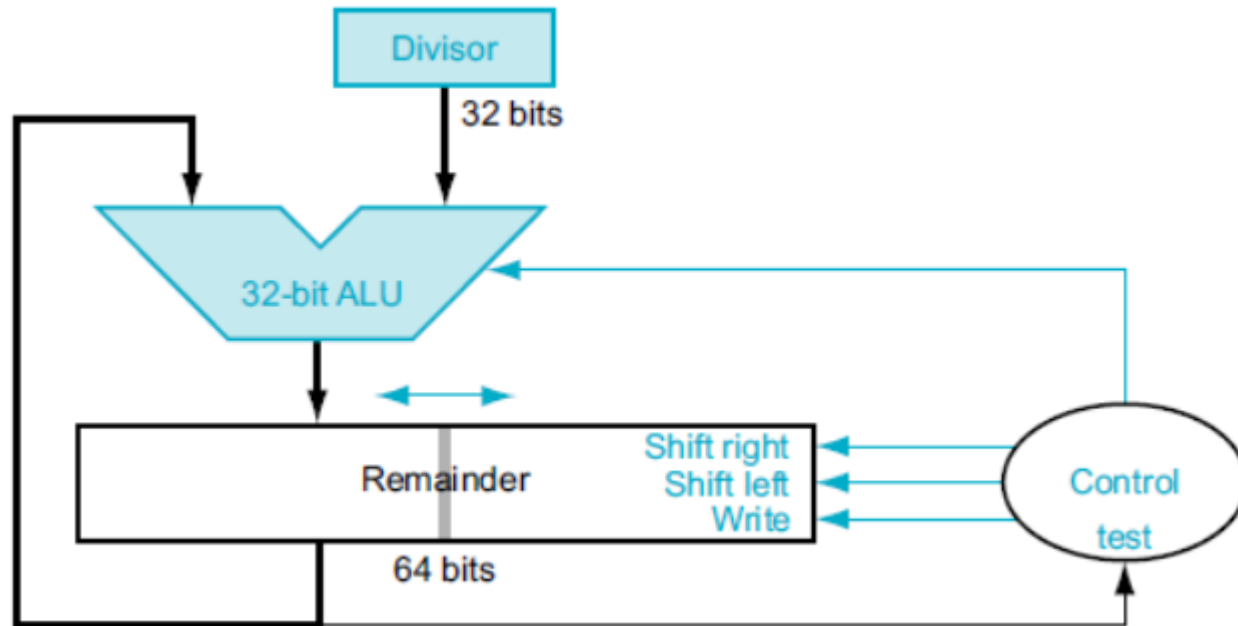
Division Hardware



Division Algorithm Example

Iteration	Step	Quotient	Divisor	Remainder
0	Initial values	0000	0010 0000	0000 0111
1	1: Rem = Rem - Div	0000	0010 0000	①110 0111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0010 0000	0000 0111
	3: Shift Div right	0000	0001 0000	0000 0111
2	1: Rem = Rem - Div	0000	0001 0000	①111 0111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0001 0000	0000 0111
	3: Shift Div right	0000	0000 1000	0000 0111
3	1: Rem = Rem - Div	0000	0000 1000	①111 1111
	2b: Rem < 0 $\Rightarrow$ +Div, SLL Q, Q0 = 0	0000	0000 1000	0000 0111
	3: Shift Div right	0000	0000 0100	0000 0111
4	1: Rem = Rem - Div	0000	0000 0100	①000 0011
	2a: Rem $\geq$ 0 $\Rightarrow$ SLL Q, Q0 = 1	0001	0000 0100	0000 0011
	3: Shift Div right	0001	0000 0010	0000 0011
5	1: Rem = Rem - Div	0001	0000 0010	①000 0001
	2a: Rem $\geq$ 0 $\Rightarrow$ SLL Q, Q0 = 1	0011	0000 0010	0000 0001
	3: Shift Div right	0011	0000 0001	0000 0001

Optimized Divider



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
 - Same hardware can be used for both

Signed Division

- So far, we have only looked at division of positive numbers
- How to divide signed numbers?
 - For Quotient, similar to how we did signed multiplication
 - Sign of the remainder must also be set -> must respect the basic equation relating them all
 - Anomalies avoided via following the rule that the dividend and the remainder must have the same sign!

Faster Division

- Can't use parallel hardware as in multiplier
 - Subtraction is conditional on sign of remainder
- Faster dividers (e.g. SRT division) generate multiple quotient bits per step
 - Still require multiple steps

RISC-V Division

- Four instructions:
 - `div`, `rem`: **signed** divide, remainder
 - `divu`, `remu`: **unsigned** divide, remainder
- **Overflow and division-by-zero don't produce errors**
 - Just return defined results
 - Faster for the common case of no error

Chapter Outline

Chapter 3 - Arithmetic for Computers

- 3.1 Introduction
- 3.2 Addition and Subtraction
- 3.3 Multiplication
- 3.4 Division
- 3.5 Floating Point
- 3.6 Parallelism and Computer Arithmetic: Subword Parallelism
- 3.7 Real Stuff: Streaming SIMD Extensions in x86
- 3.8 Going Faster: Subword Parallelism and Matrix Multiply
- 3.9 Fallacies and Pitfalls
- 3.10 Concluding Remarks
- 3.11 Historical Perspective and Further Reading

Floating Point

*“Speed gets you nowhere if
you’re headed the wrong way”*

American proverb

Floating Point

- Representation for non-integral numbers
 - Including very small and very large numbers
- Like scientific notation
 - -2.34×10^{56} ← normalized
 - $+0.002 \times 10^{-4}$ ← not normalized
 - $+987.02 \times 10^9$ ← not normalized
- In binary
 - $\pm 1.xxxxxxx_2 \times 2^{yyyy}$
- Types float and double in C

Floating Point Representation

- Tradeoff between precision (fraction part) and range (exponent part)
 - > Good design demands good compromise!
- Different levels of range and precision
 - Half precision (16 bits)
 - Single Precision (32 bits) – float
 - Range = 2×10^{-38} to 2×10^{38}
 - Extraordinary but not infinity -> overflow still possible!
 - And underflow!



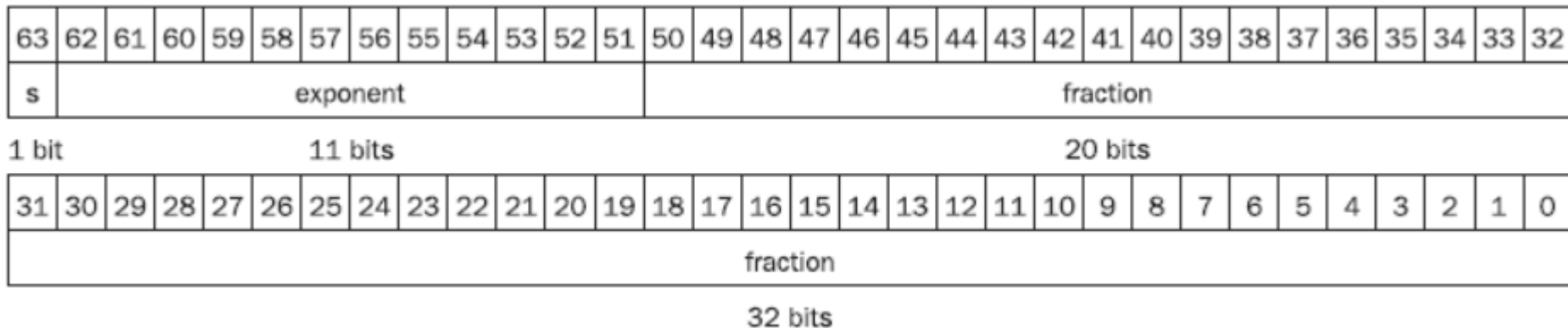
Floating Point Representation

- Different levels of precision

- Double Precision (64 bits) – double

- Range = 2×10^{-308} to 2×10^{308}

- Primary advantage is not the increased range but rather the enhanced precision!



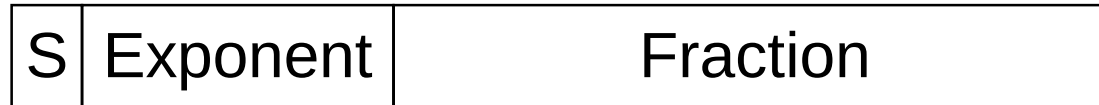
Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
 - Portability issues for scientific code
- Now almost universally adopted
- Two representations
 - Single precision (32-bit)
 - Double precision (64-bit)

IEEE Floating-Point Format

single: 8 bits
double: 11 bits

single: 23 bits
double: 52 bits



- Equivalent decimal number
- Normalize significand:
- $1.0 \leq |\text{significand}| < 2.0$
- Exponent: biased representation: actual exponent + Bias
 - Ensures exponent is unsigned
 - Single: Bias = 127; Double: Bias = 1203

IEEE Floating-Point Format

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

- Exponents 00000000 and 11111111 reserved
- Smallest value ?
- Largest value ?